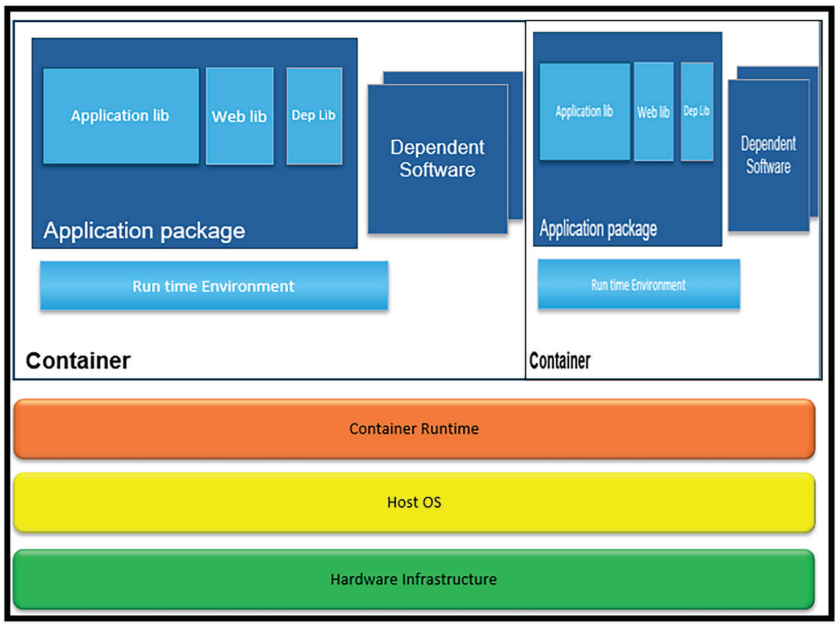

Figure 1: Container environment

to run. They encapsulate microservices code, dependencies, libraries, binaries, and more, providing a high degree of isolation, as well as fast initialisation and execution.

Containerisation ensures consistency across development, testing and production environments, as containers behave the same regardless of where they are deployed. They leverage orchestrators to manage microservices-based applications using container tools. The most popular frameworks to orchestrate multiple containers in an enterprise environment are Docker and Kubernetes.

Containers provide a minimal footprint, allowing for faster startup times and efficient resource utilisation. This is essential for rapid scaling and maintaining a low mean time to recovery (MTTR) during incidents.

Once built, container images are stored in container registries (such as Docker Hub or private registries). These registries act as a source of truth for deploying the most current versions of the microservices.

Container features include:

- Ability to reduce memory footprint to bare minimum.
- With a container image, we can bundle the application along with its runtime and dependencies. This makes the application very portable.
- Containers are very lightweight. They use less memory and CPU than VMs running similar workloads.
- They can be built locally, deployed and run anywhere, either in the cloud or on-premises.
- Containerised services can be deployed very fast (within milliseconds). They can be scaled up or down with ease.

The need for container orchestration

Orchestration is about automating deployment and operational efforts when managing containerised microservices. Key considerations are:

- Deploying for scalability and availability.
- Implementing automated health checks and monitoring.
- Setting resource limits.
- Updating the orchestration platform for critical security notifications and patches released by the platform provider.
- Automating backup for faster local replicability.
- Implementing business continuity planning.

Kubernetes, DockerSwarm and Apache Mesos are popular open source container orchestration solutions.

Kubernetes

Kubernetes is an orchestration framework for deploying, scaling, and managing containers. It automates the deployment of containerised microservices.

Key Kubernetes components include:

Pods: This is the smallest deployable unit that can house one or more containers. Pods are not expected to live long. Persistent volume can be assigned and shared between pods in the same node.

Deployments: These define the desired state (e.g., the number of pod replicas) and manage rolling updates to ensure availability.

Services: These provide stable network endpoints and load balancing, enabling reliable service discovery for inter-service communication.

Infrastructure features: Kubernetes offers built-in capabilities like auto-scaling, self-healing (rescheduling failed pods), and configuration management (using ConfigMaps and Secrets), which are crucial for running microservices at scale reliably.

Its features are:

- Quick deployment of applications.
- Portable — can be used with public, private, hybrid and multi-cloud.
- Extensible — can be modular, pluggable, hookable and composable.
- Automated deployment and replication of containers.
- Online scale-in or scale-out of applications on-the-fly.
- Load balancing over a group of containers.
- Rolling view features of application containers.
- Resilience due to automated